

Intro to probabilistic training

Mariana Clare

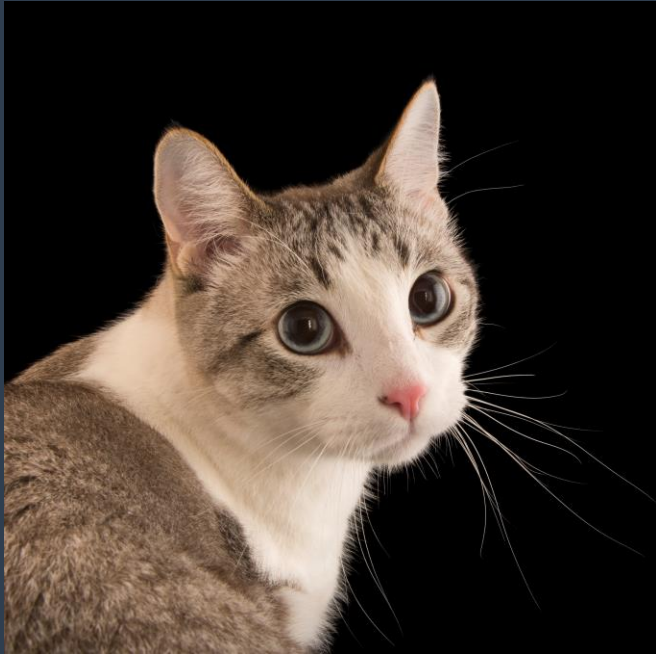
Outline

- Why is probabilistic training important?
- Different types of methods to quantify uncertainty
- Probabilistic training in anemoi

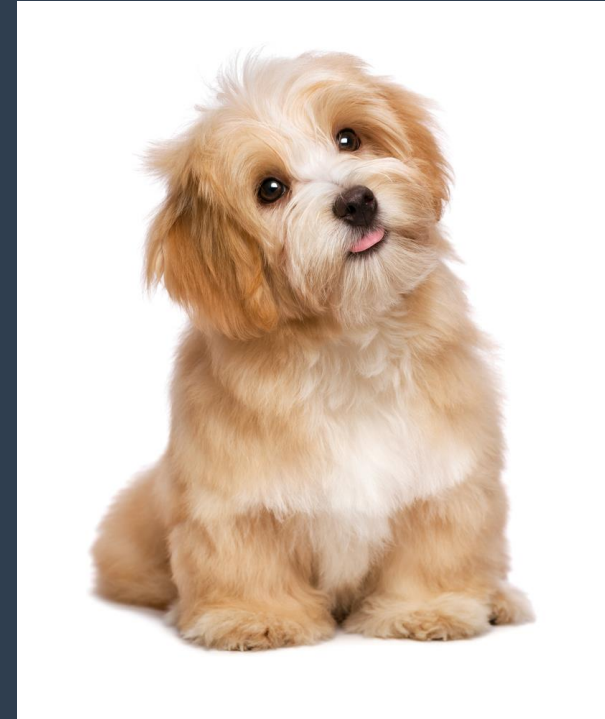
Why is quantifying uncertainty important?

What happens if we have no uncertainty in our prediction?

Neural Network output:



CAT



NOT CAT

But this has limited value...

For machine learning methods to be trustworthy, they to be able to tell us when they're *not sure*

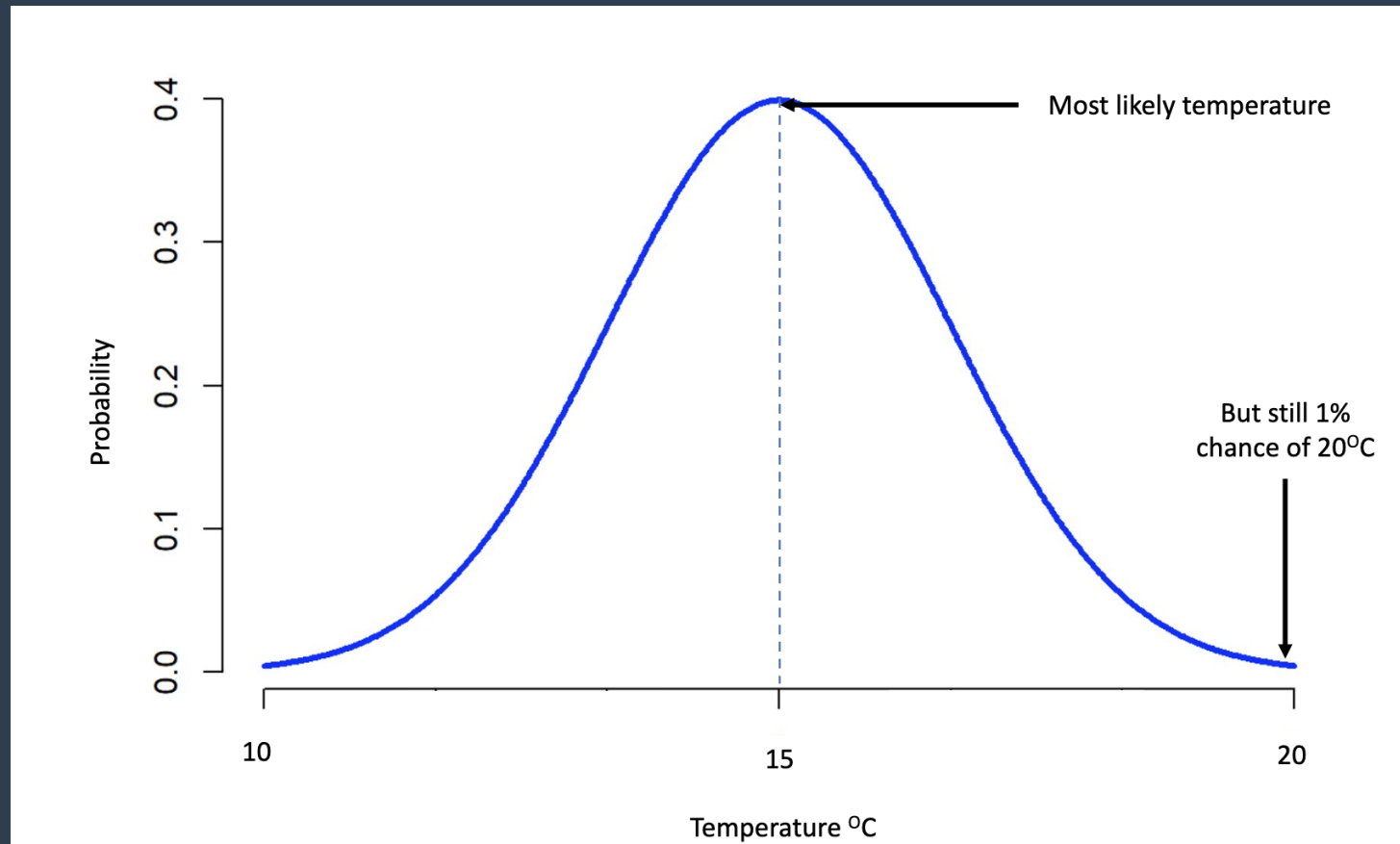
This is especially true for eather and climate forecasts, which are inherently uncertain because of:

- Chaotic nature of Earth System
- Uncertainty in Input Conditions
- Uncertainty in Model itself

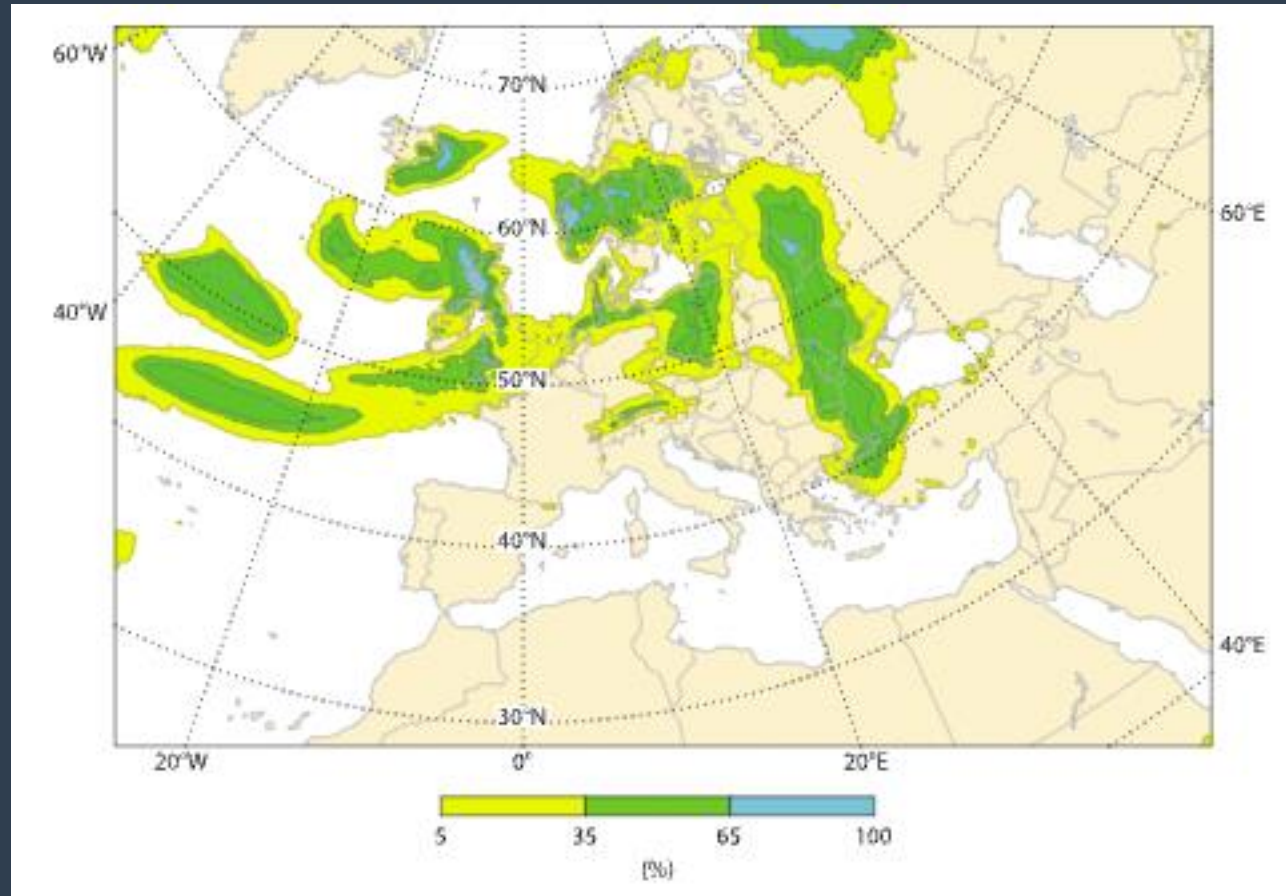
and is particularly important for extreme events when we need to know if there is even a slight chance of one occurring

But this has limited value...

Instead of the temperature in Reading in 4 days time will be 15°C (likely wrong), we need:



In weather models, we quantify this uncertainty using an ensemble forecast with perturbed initial conditions and perturbations in model itself



Ensemble-based probability of precipitation in excess of 5mm

Aleatoric vs. Epistemic Uncertainty

Epistemic Uncertainty:

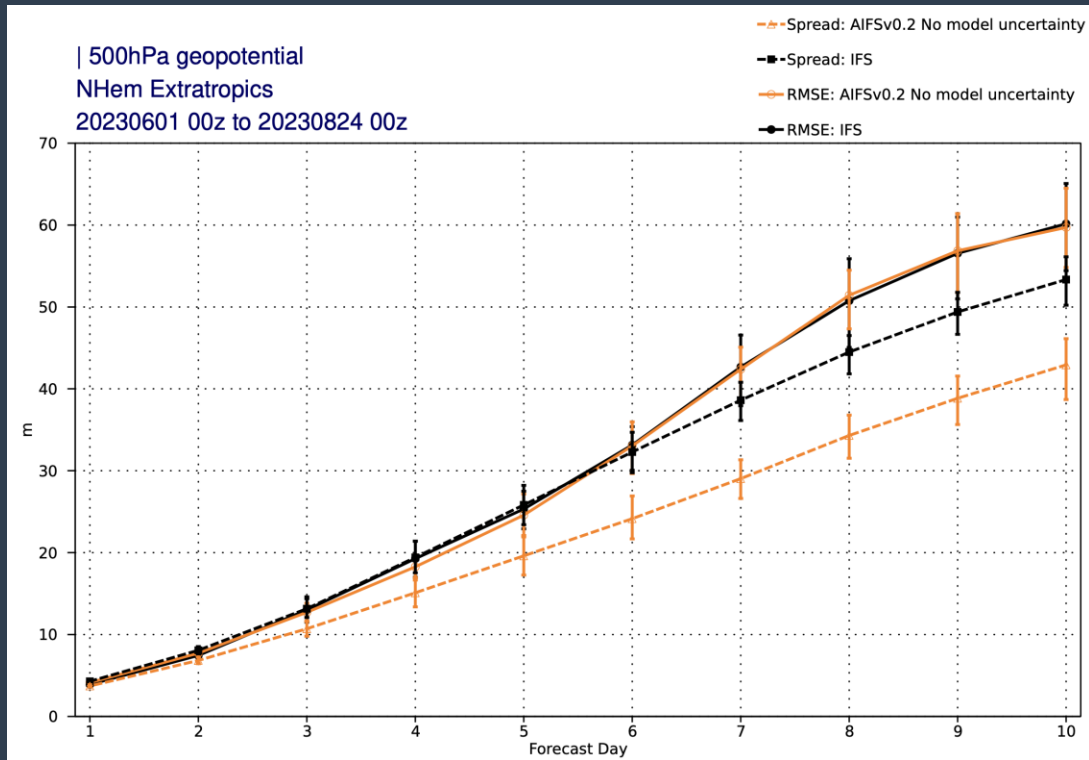
- Uncertainty in the model
- Possible to reduce with more data
- Quantify using an ensemble of models

Aleatoric Uncertainty:

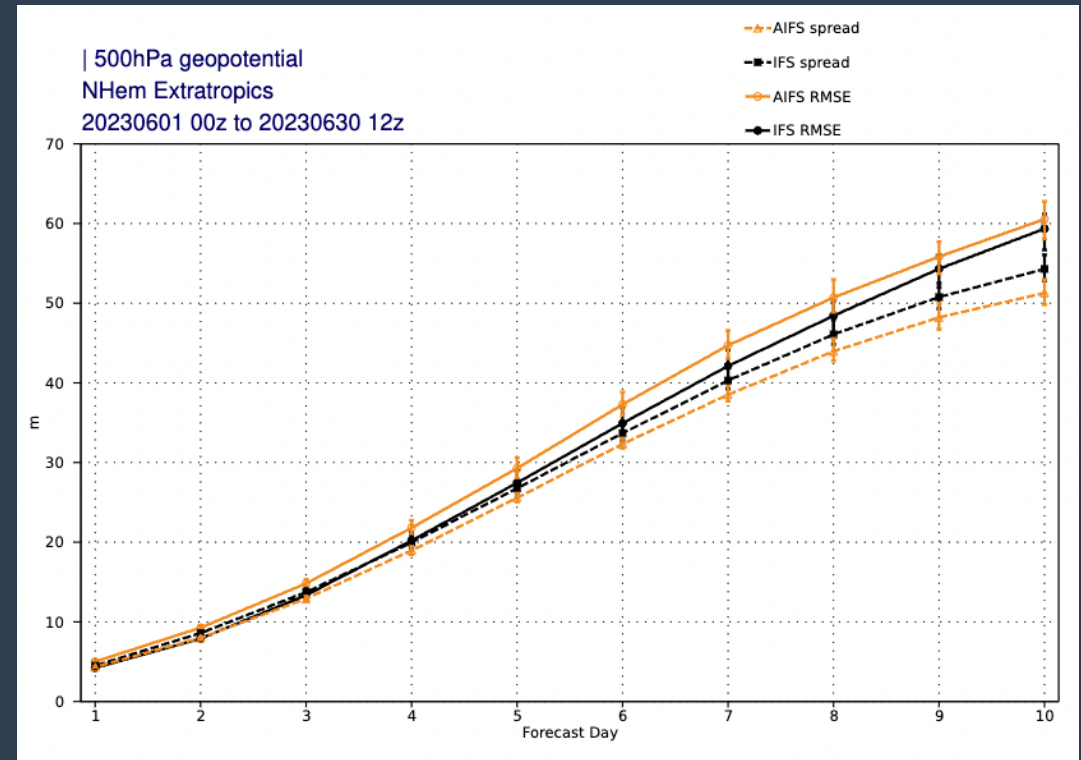
- Uncertainty in input data
- Irreducible uncertainty but still needs to be quantified
- Quantify by perturbing input data and/or predicting a distribution rather than single output

Variance-Error relationships

To quantify uncertainty properly, we need to quantify both aleatoric and epistemic



Only aleatoric uncertainty

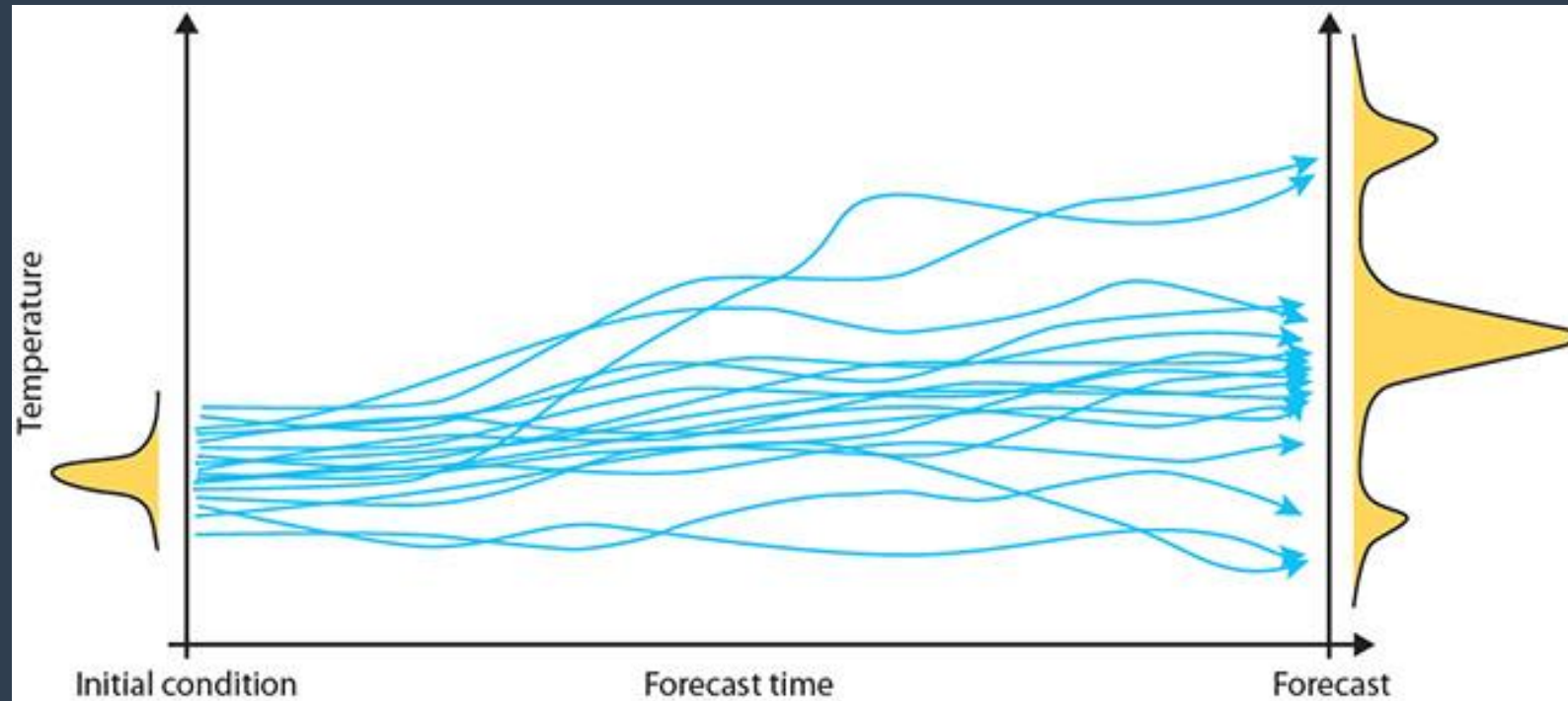


Aleatoric and epistemic uncertainty

**What if all we have is a model
checkpoint?**

Test-time augmentation

Run model predictions multiple times perturbing initial conditions and propagating uncertainty to the outputs



Results can be improved by making sensible choices for initial condition perturbations

This is similar to how we estimate aleatoric uncertainty in the IFS ensemble

Test-time augmentation

Advantages:

- Very easy to implement
- In an ML model, this is not too expensive because prediction is cheap

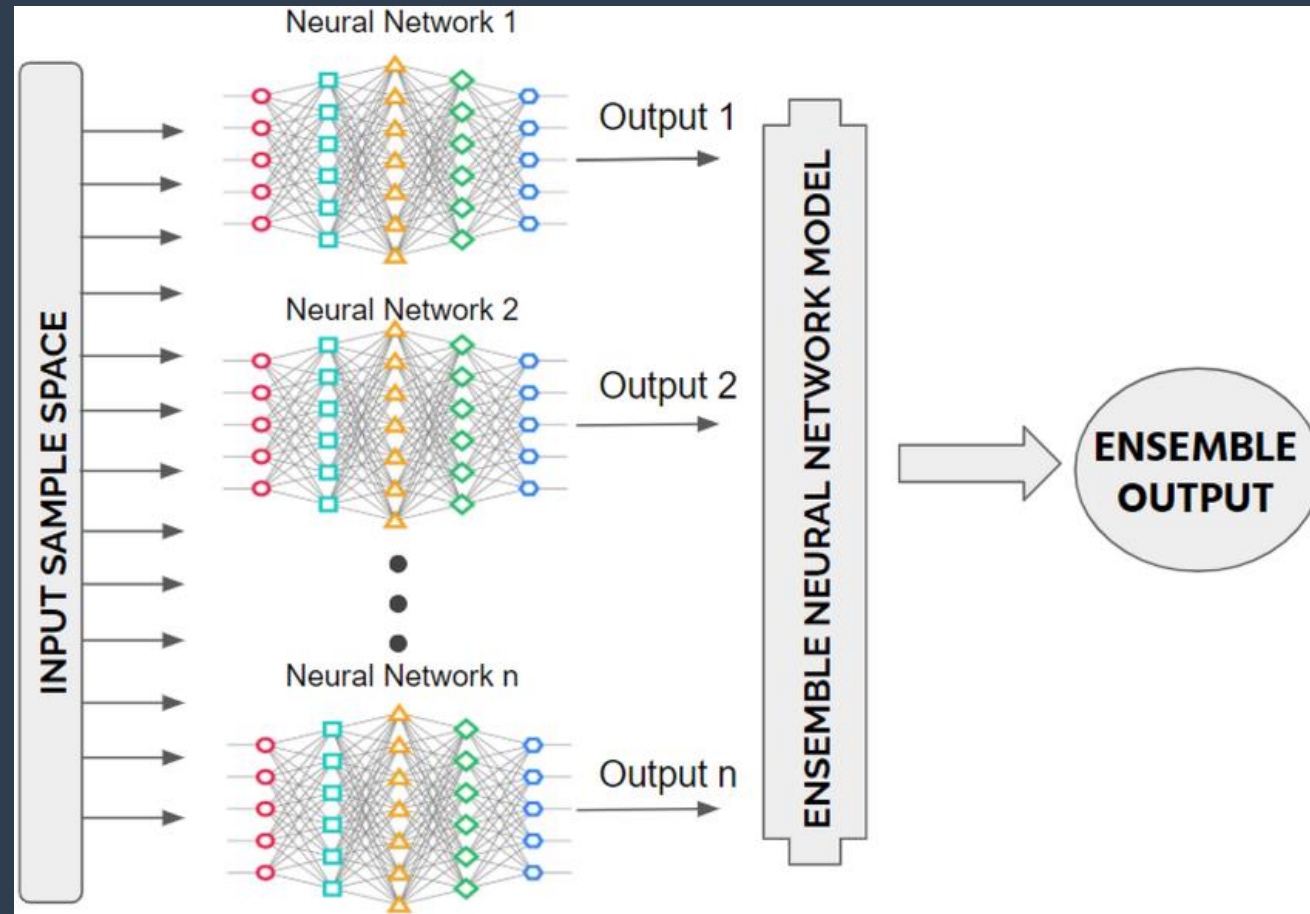
Disadvantages:

- Can be difficult to generate sensible/realistic perturbations
- Only quantifies aleatoric uncertainty

Can we just train 50 models?

Deep ensembles

Train multiple (deep) Neural Networks and use them to create an ensemble of models. These neural networks have different architectures and can even be different types of model



Deep ensembles

Advantages:

- Easy to implement without changing any of the training procedure
- Capture epistemic uncertainty and can be easily combined with test-time augmentation to capture aleatoric uncertainty too

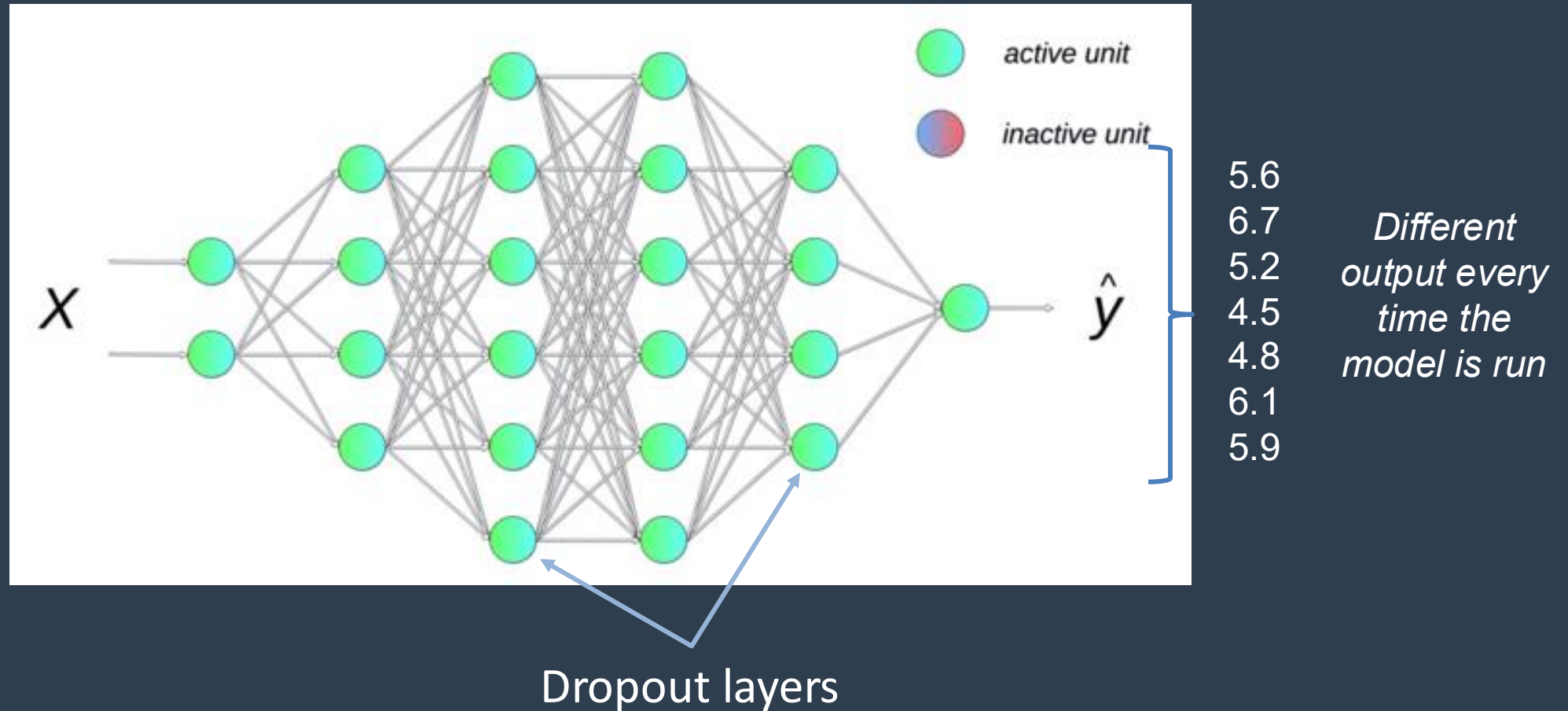
Disadvantages:

- Expensive because have to train multiple models (but can be parallelised)
- The models in the ensemble will have very different levels of accuracy and it can be non-trivial to combine them into a coherent distribution after

What if we don't have access to a huge supercomputer?

Dropout layers

Dropout layers were originally designed to stop overfitting during training, but they can be used during the inference stage to generate an ensemble of outputs



Dropout layers

Advantages:

- Very easy to implement
- Has the advantage of also preventing overfitting in training

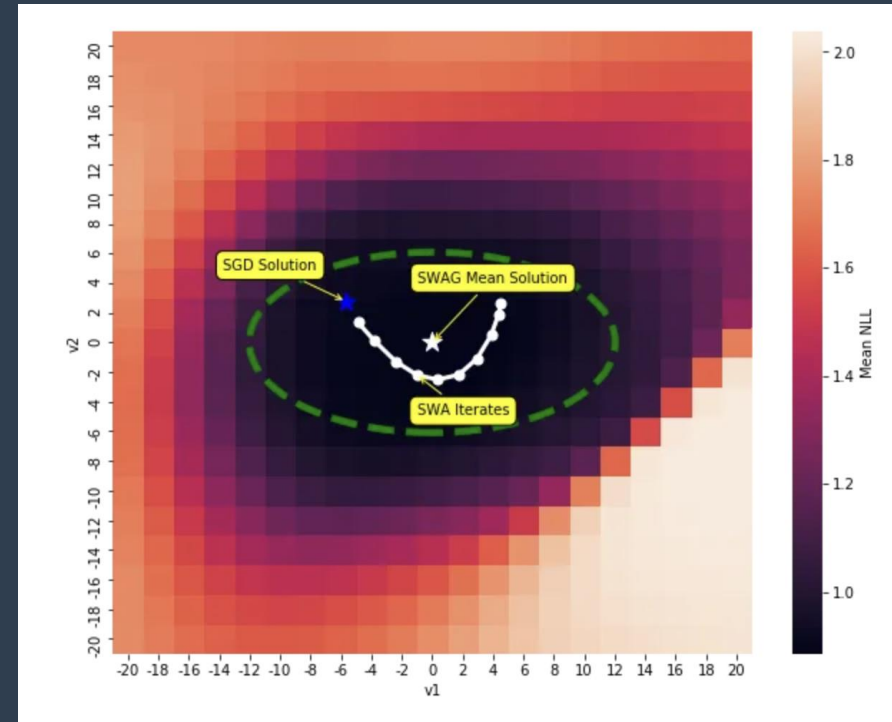
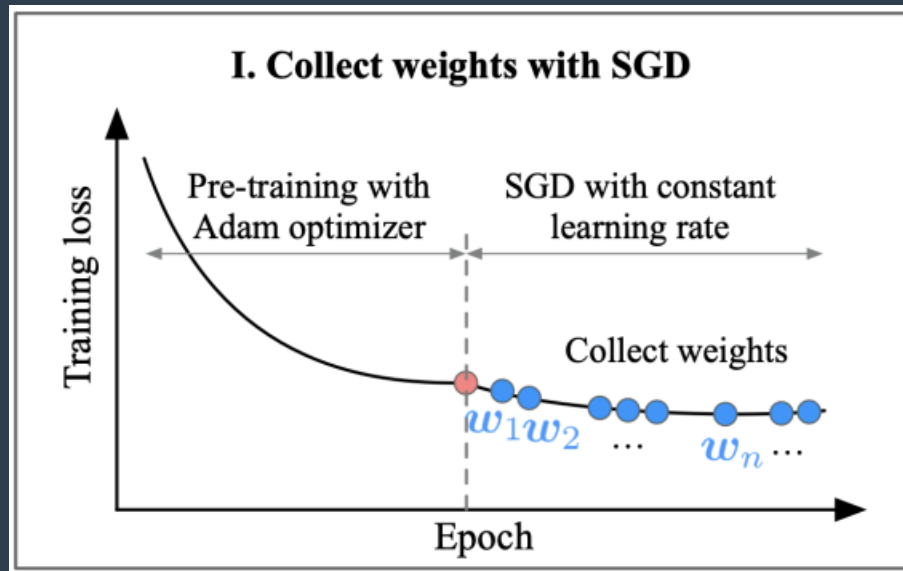
Disadvantages:

- Only quantifies epistemic uncertainty
- Often underestimates uncertainty of the model (underspread)

Stochastic Weighted Averaging (SWAG)

During training of a NN, when the optimum has almost been reached, the learning rate is pushed to explore the local geometry using Stochastic Gradient Descent (SGD).

At each epoch, a new model solution is obtained (generating) an ensemble of models and the SWA mean solution is closer to the optimum solution than the SGD one



SWAG

Advantages:

- Uses same training procedure as a deterministic neural and has same computational cost
- Creates an ensemble of models that can be combined with test-time augmentation to quantify both epistemic and aleatoric uncertainty

Disadvantages:

- Only works if the uncertainty in the gradient descent algorithm is gaussian
- Often underestimates the uncertainty of the problem

**Can we capture uncertainty by
using probabilistic loss functions?**

Loss functions and entropy for probabilistic predictions

If the output is probabilistic (i.e. a distribution), then standard loss functions cannot be used.

It is common practice to instead minimise the KL-divergence loss function (difference between the real distribution and the predicted distribution)

$$D_{KL}(q||p) = \int_W q(W') \log \left(\frac{q(W')}{p(W'|D_{tr})} \right) dW'$$

For probabilistic predictions, the entropy, H_i , can also be calculated which is a measure of the prediction uncertainty

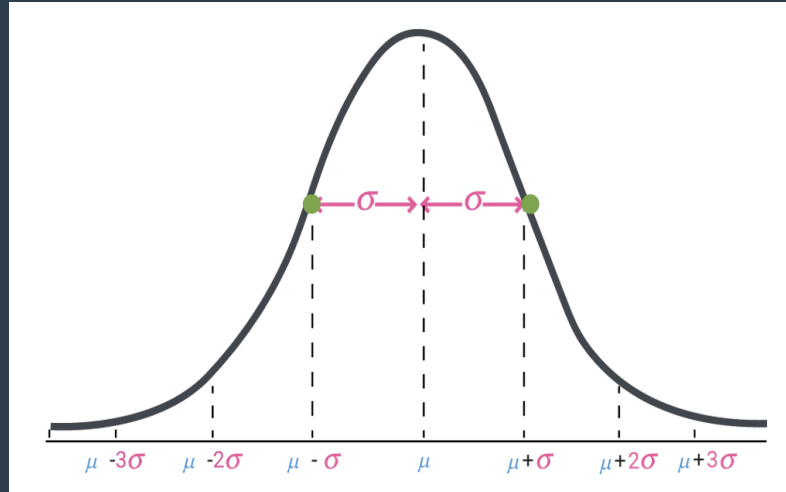
$$H_i = - \sum_{j=1}^{N_i} p_{ij} \log(p_{ij})$$

where N_i is the number of possible outcome and p_{ij} is the probability of each outcome j for sample i .

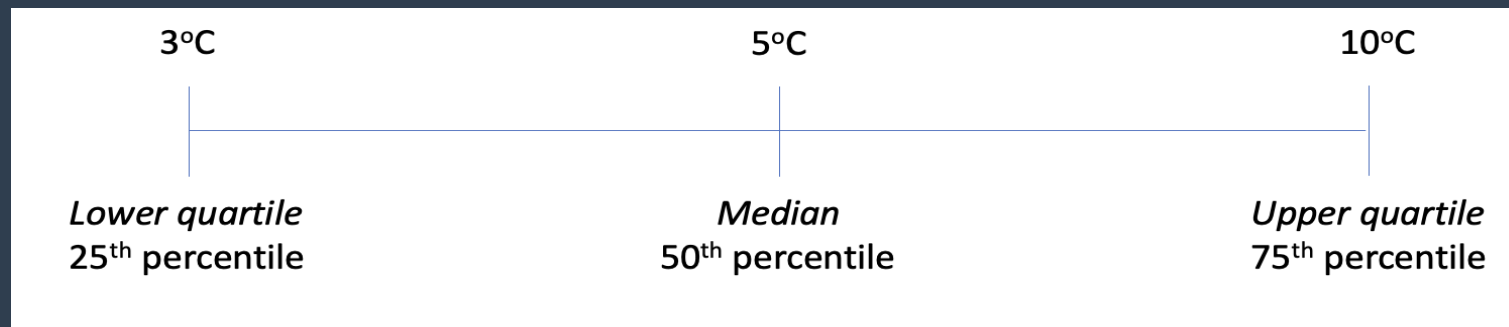
The larger the entropy the more uncertain the model is of the result

Predict a parametric distribution

Instead of predicting deterministic output, instead, for example, predict mean μ and variance σ that define a *normal distribution*



Or, for example, predict quantiles of a distribution



Predicting a parametric distribution

Advantages:

- Easy to train and training costs are roughly equivalent to those for a deterministic neural network
- Can be applied to any deep learning neural network method

Disadvantages:

- Requires knowledge of the output distribution which is not always known and assumptions of its shape
- In some cases, we require ensemble members rather than distributions for downstream applications

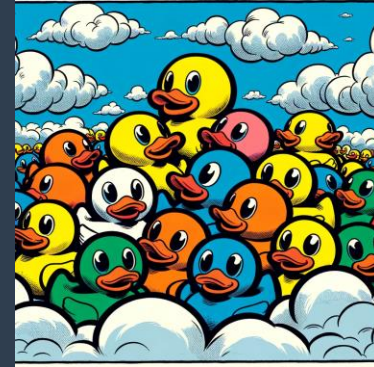
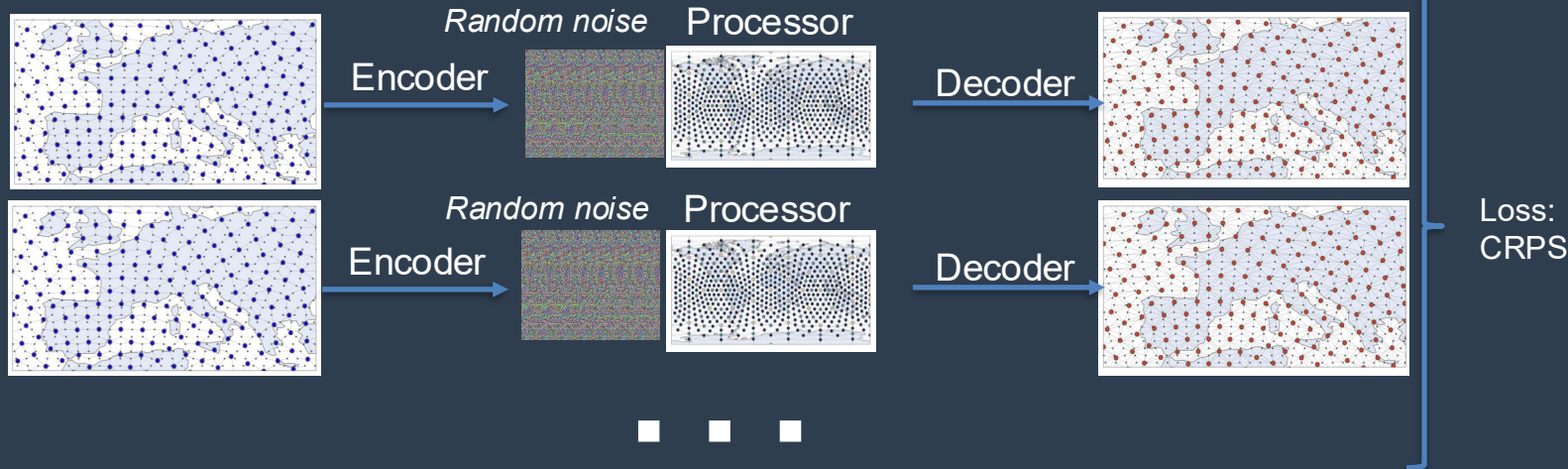
What if we don't have an output distribution?

Probabilistic training in Anemoi

Ensemble forecasts ...

- a) Instead of a MSE loss, learn an ensemble via optimizing probabilistic scores

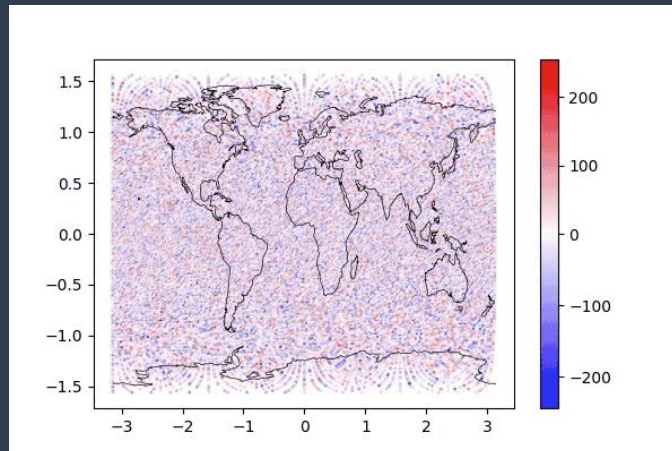
Ensemble initial conditions



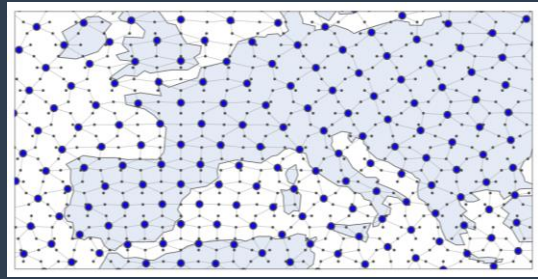
- b) Create a forecast as de-noising task (diffusion training)

for example:

- Stable diffusion -> Images
- Sora -> Video
- Gencast -> Weather
(Price et al. (2023))



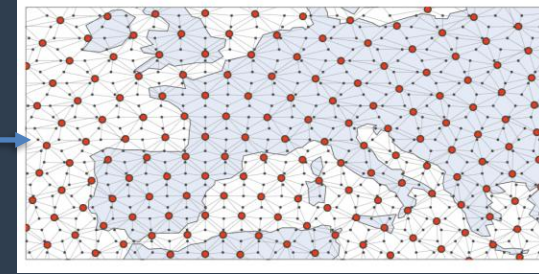
Injecting Noise



Encoder

Processor

Decoder



Injecting Noise

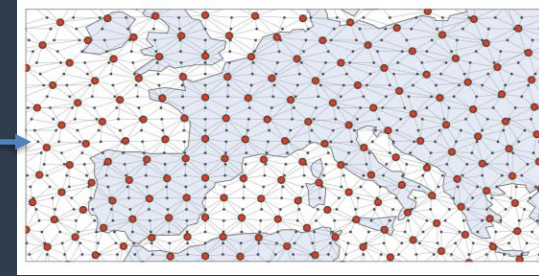
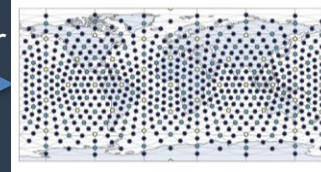
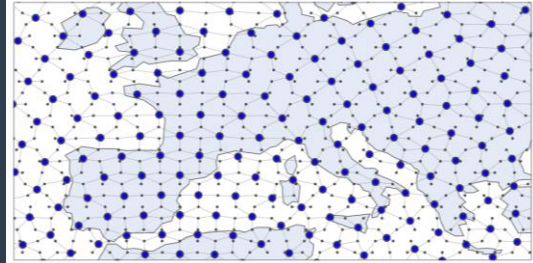
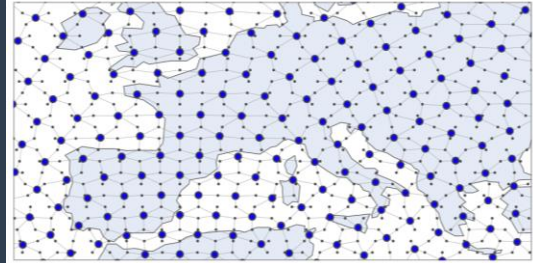
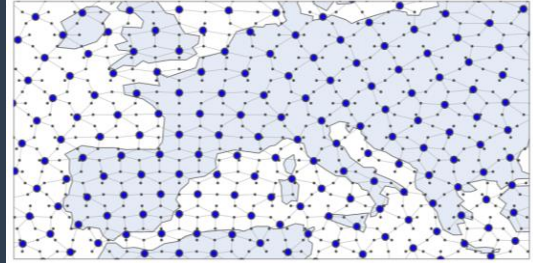
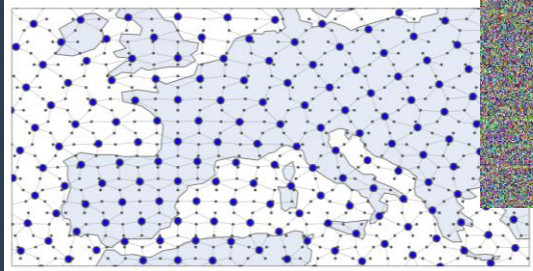
Random noise



Encoder

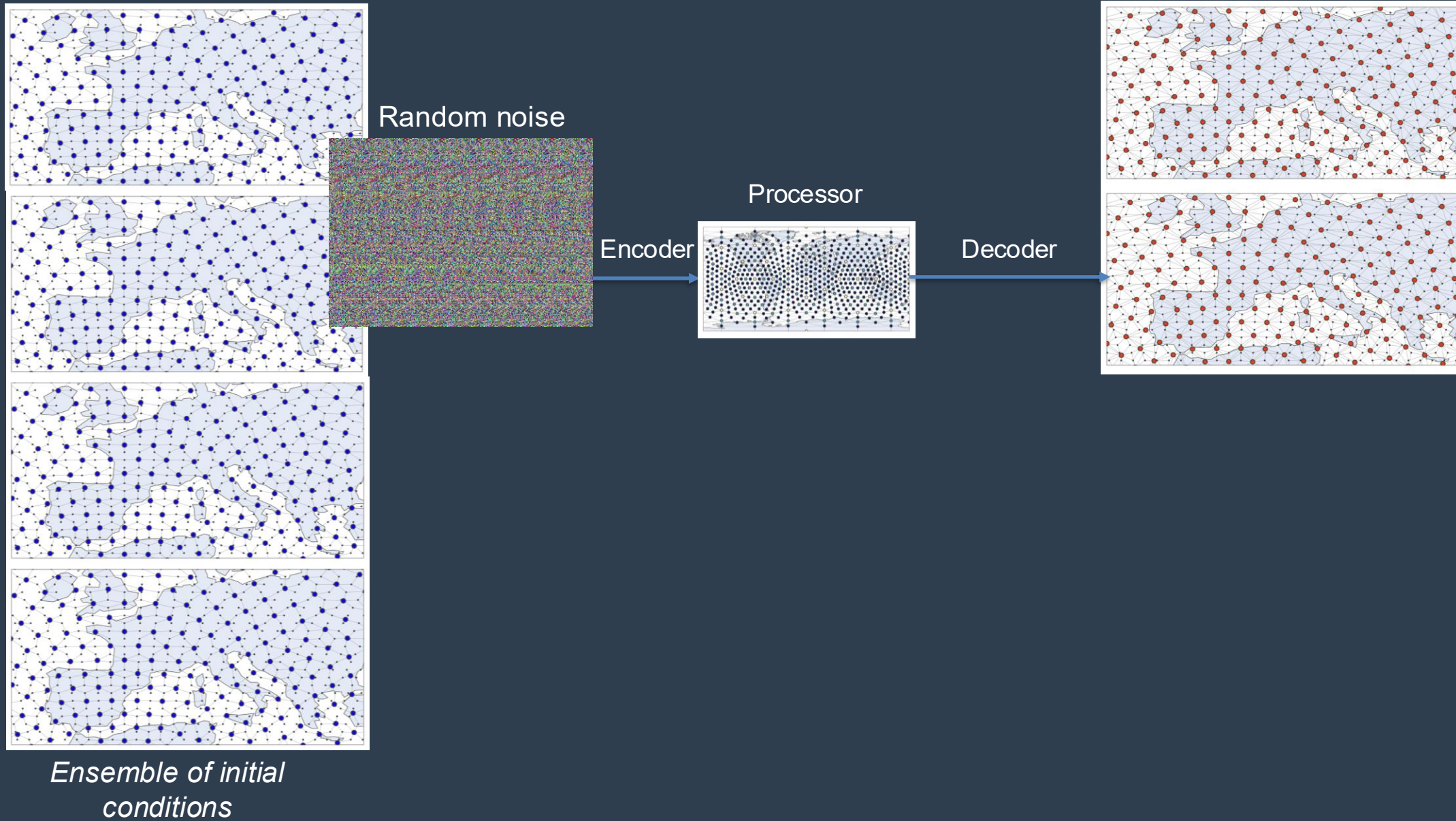
Processor

Decoder

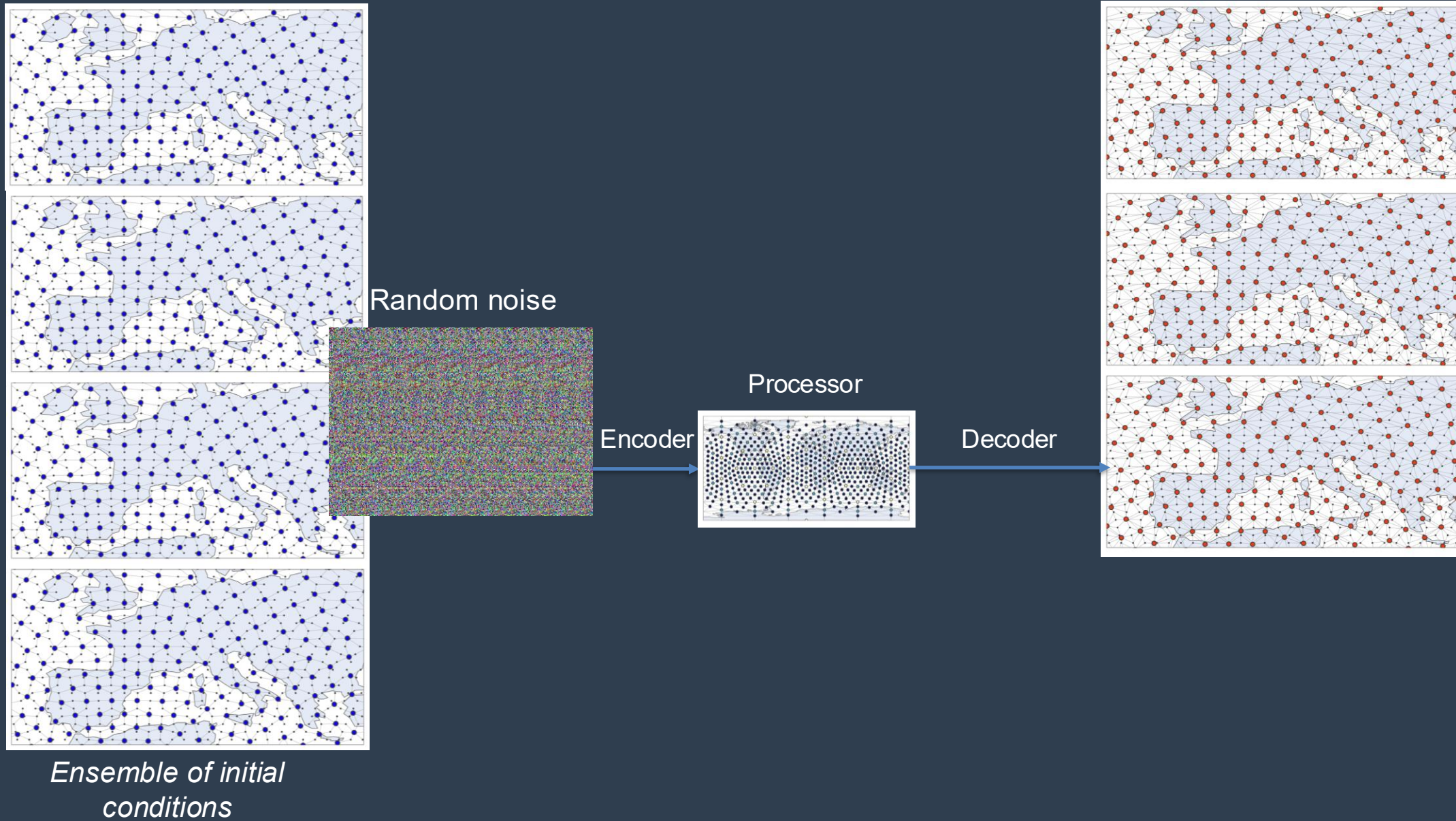


*Ensemble of initial
conditions*

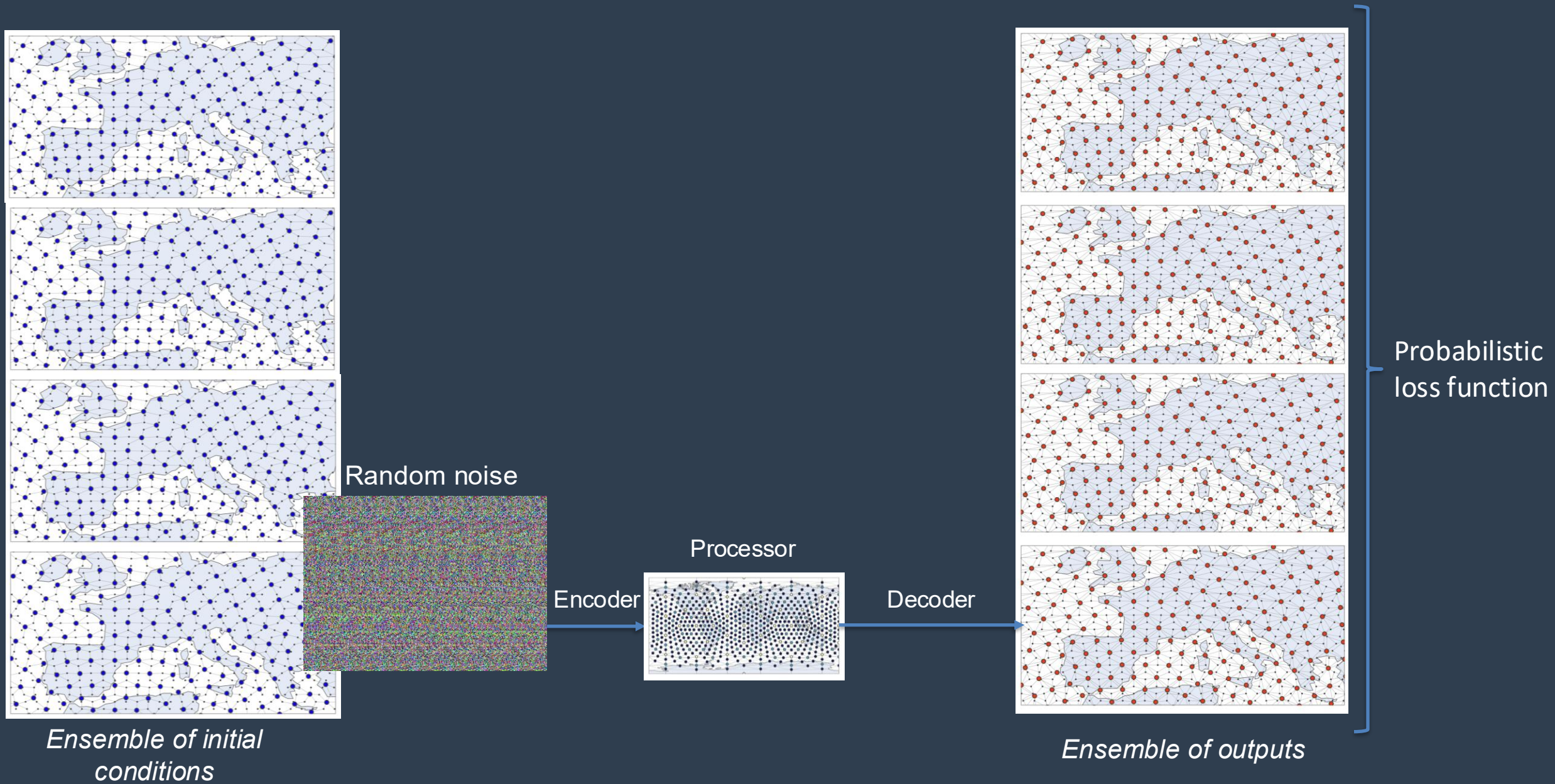
Injecting Noise



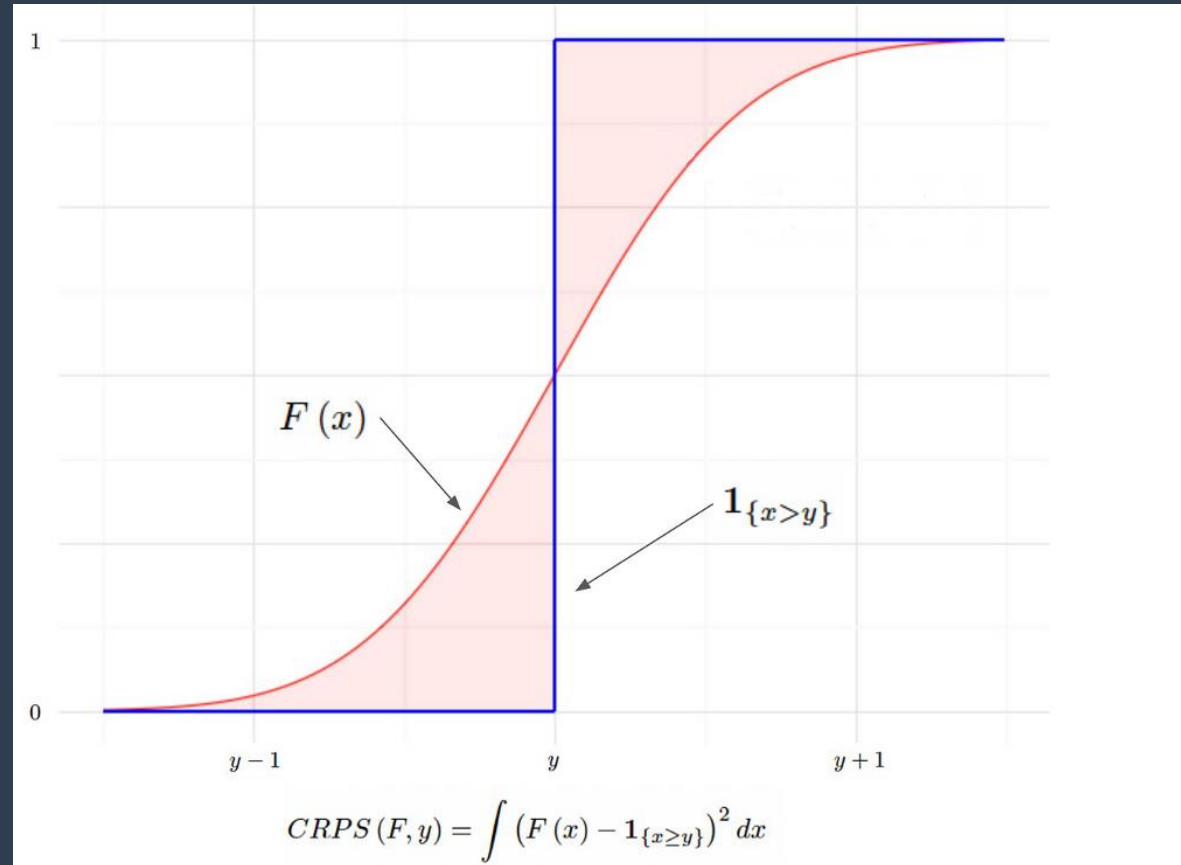
Injecting Noise



Injecting Noise

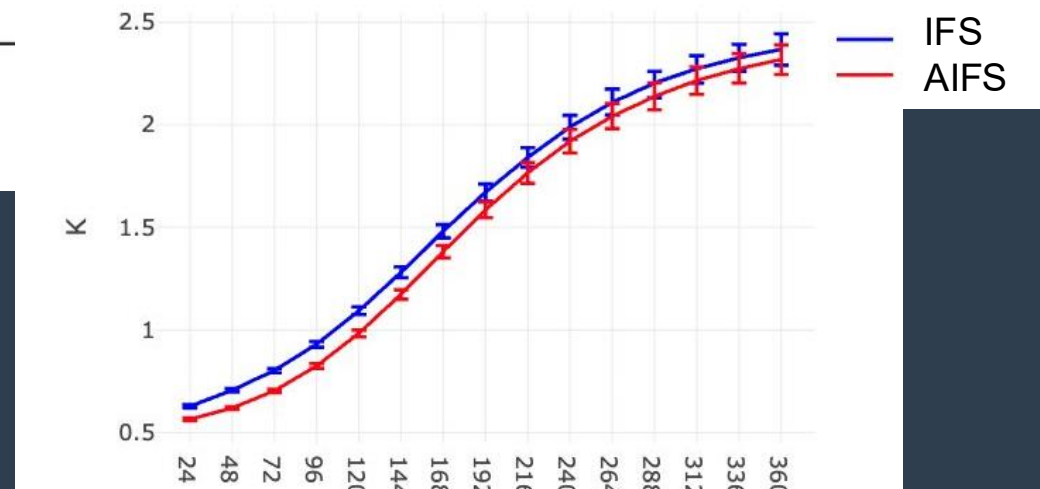
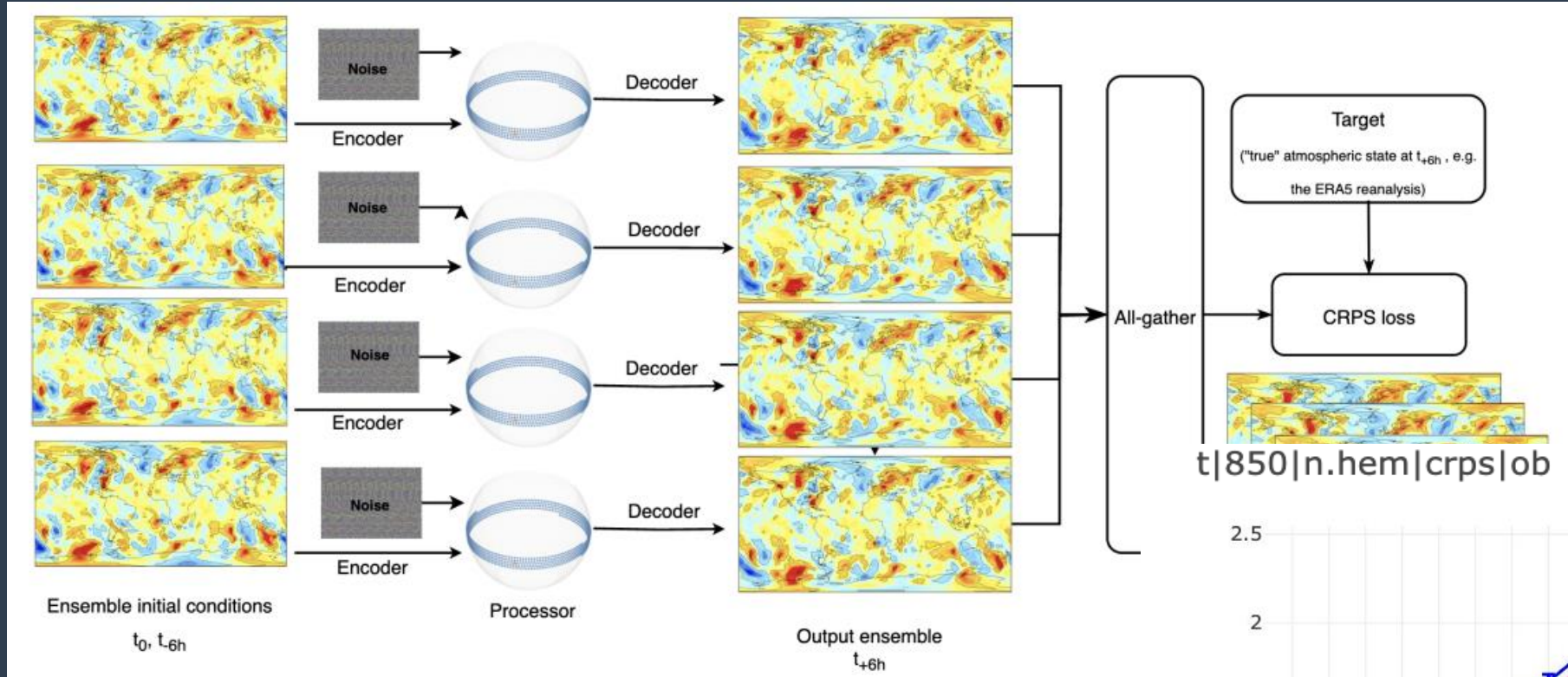


Continuous Ranked Probability Score (CRPS)



AIFS CRPS

Lang et al 2024b
Operational system from 1/7/25



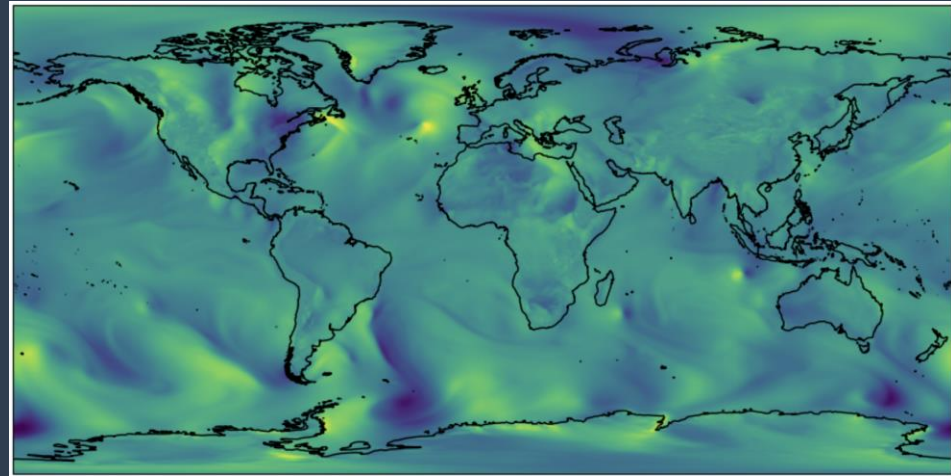
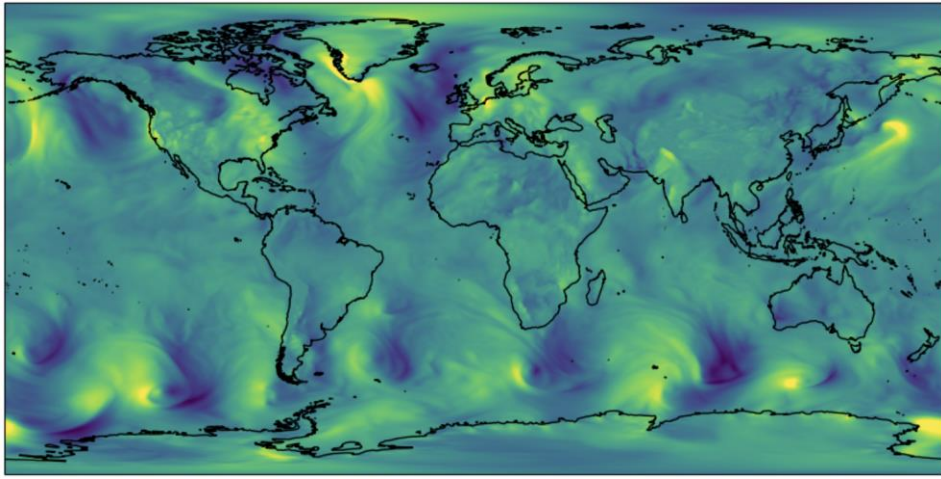
AIFS CRPS: CRPS training loss avoids smoothed forecasts

meridional wind at 850 hPa

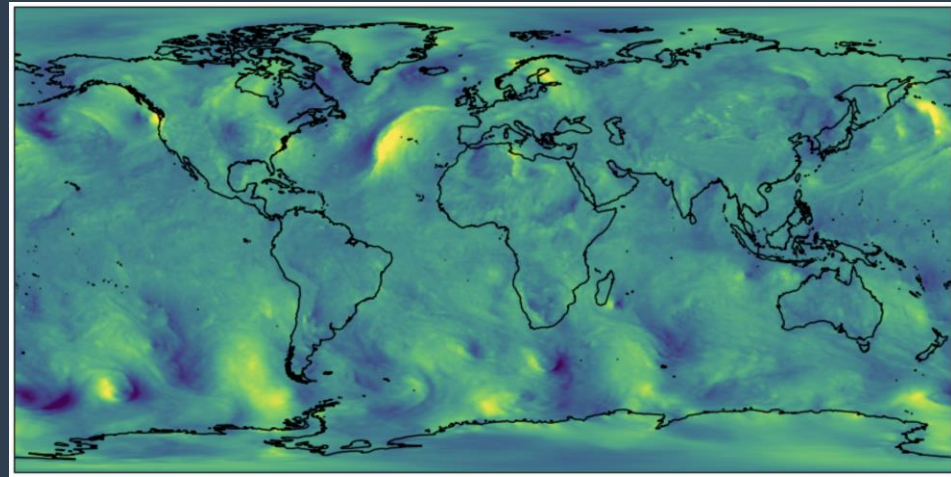
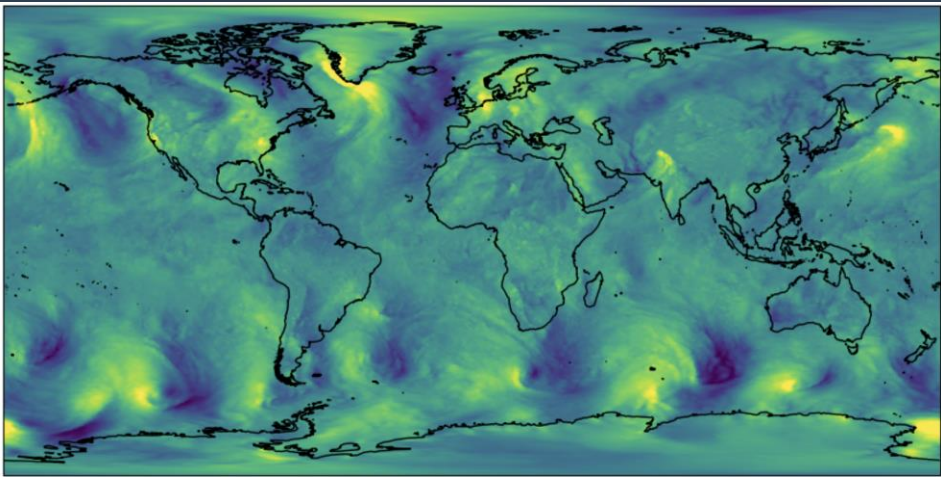
1-day forecast

10-day forecast

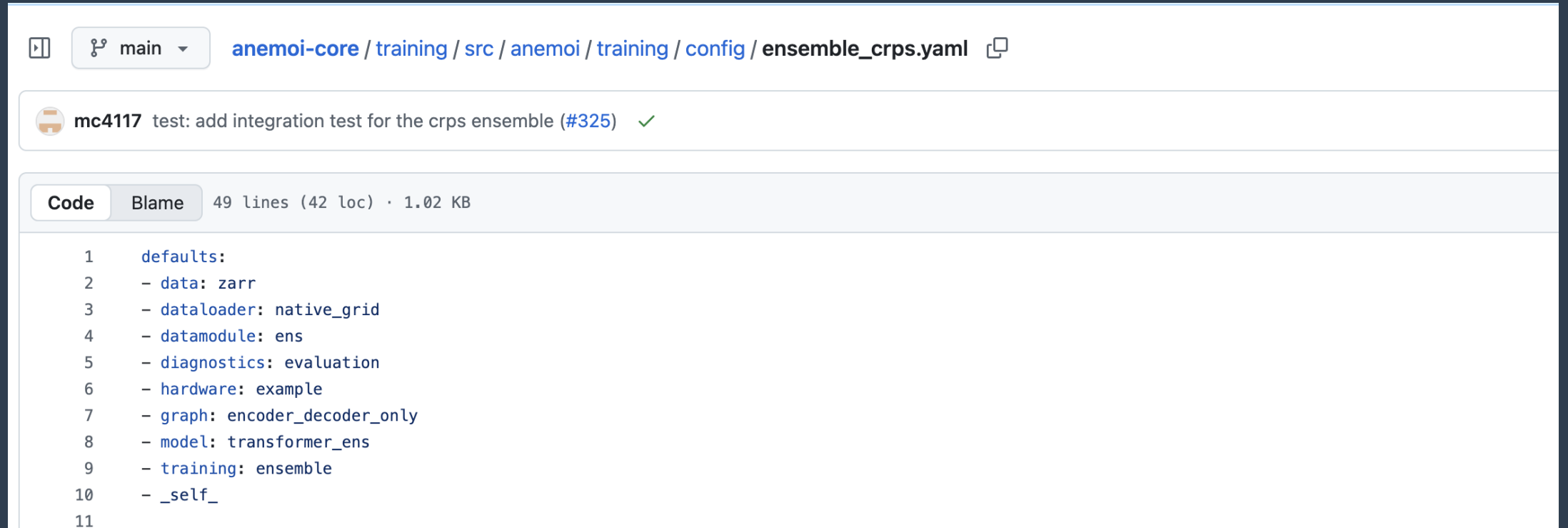
AIFS Single



AIFS CRPS



Running in Anemoi



The screenshot shows a GitHub interface. At the top, the repository path is `anemoi-core / training / src / anemoi / training / config / ensemble_crps.yaml`. Below this, a pull request is displayed: `mc4117` with the title `test: add integration test for the crps ensemble (#325)` and a green checkmark indicating it is merged. The file `ensemble_crps.yaml` is selected, showing 49 lines (42 loc) and 1.02 KB. The code content is as follows:

```
1 defaults:
2   - data: zarr
3   - dataloader: native_grid
4   - datamodule: ens
5   - diagnostics: evaluation
6   - hardware: example
7   - graph: encoder_decoder_only
8   - model: transformer_ens
9   - training: ensemble
10  - _self_
11
```

Documentation: <https://anemoi.readthedocs.io/projects/training/en/latest/user-guide/kcrps-set-up.html>

Injection and shaping of noise

Advantages:

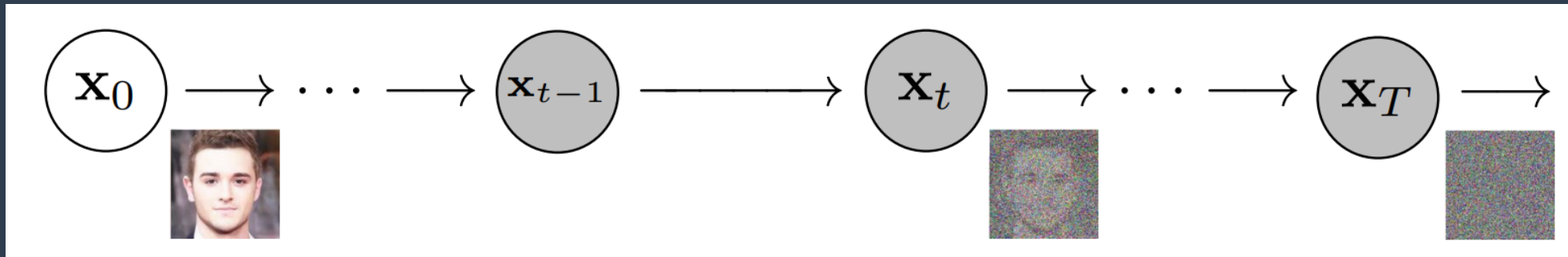
- Simple to implement if the correct loss function is chosen, because the neural network itself learns the uncertainties (and Anemoi allows flexible choice of loss function)
- Relatively cheap to implement
- Can be combined with an ensemble of initial conditions in training and inference

Disadvantages:

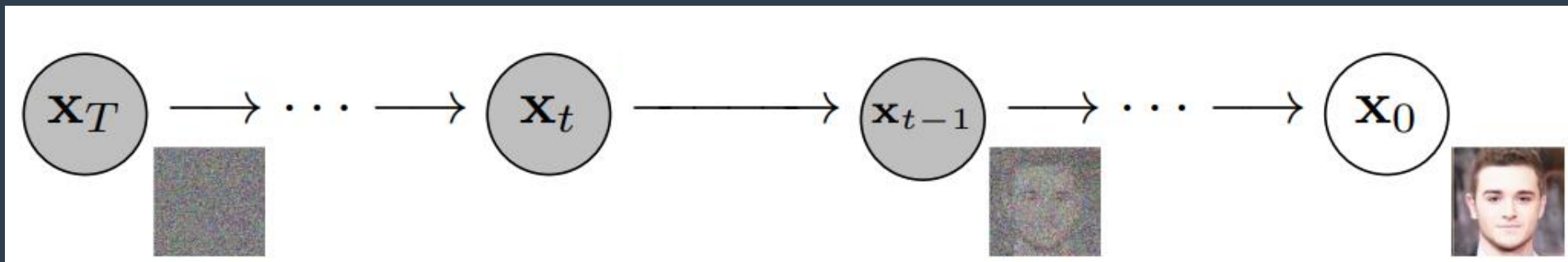
- Can increase the memory usage in training (although for a big problem this memory increase is insignificant)
- Small scale artifacts can appear but these can be countered by multi-scale loss function (implemented in Anemoi)

Diffusion Model

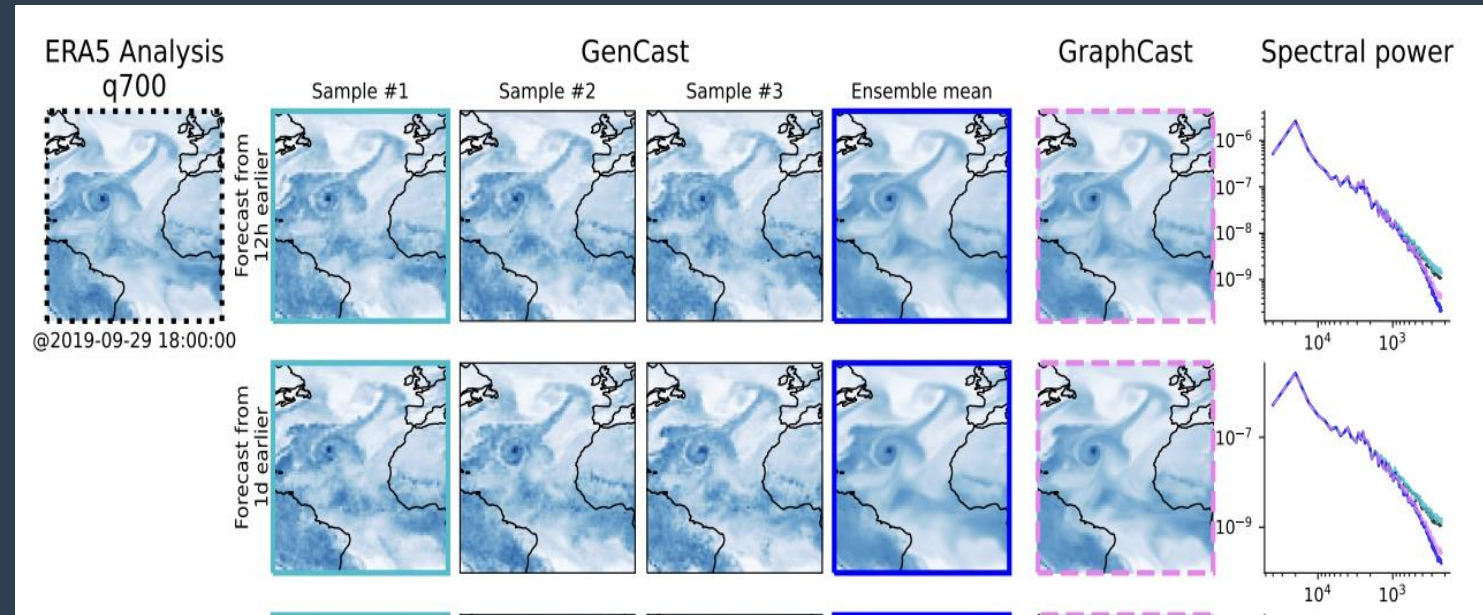
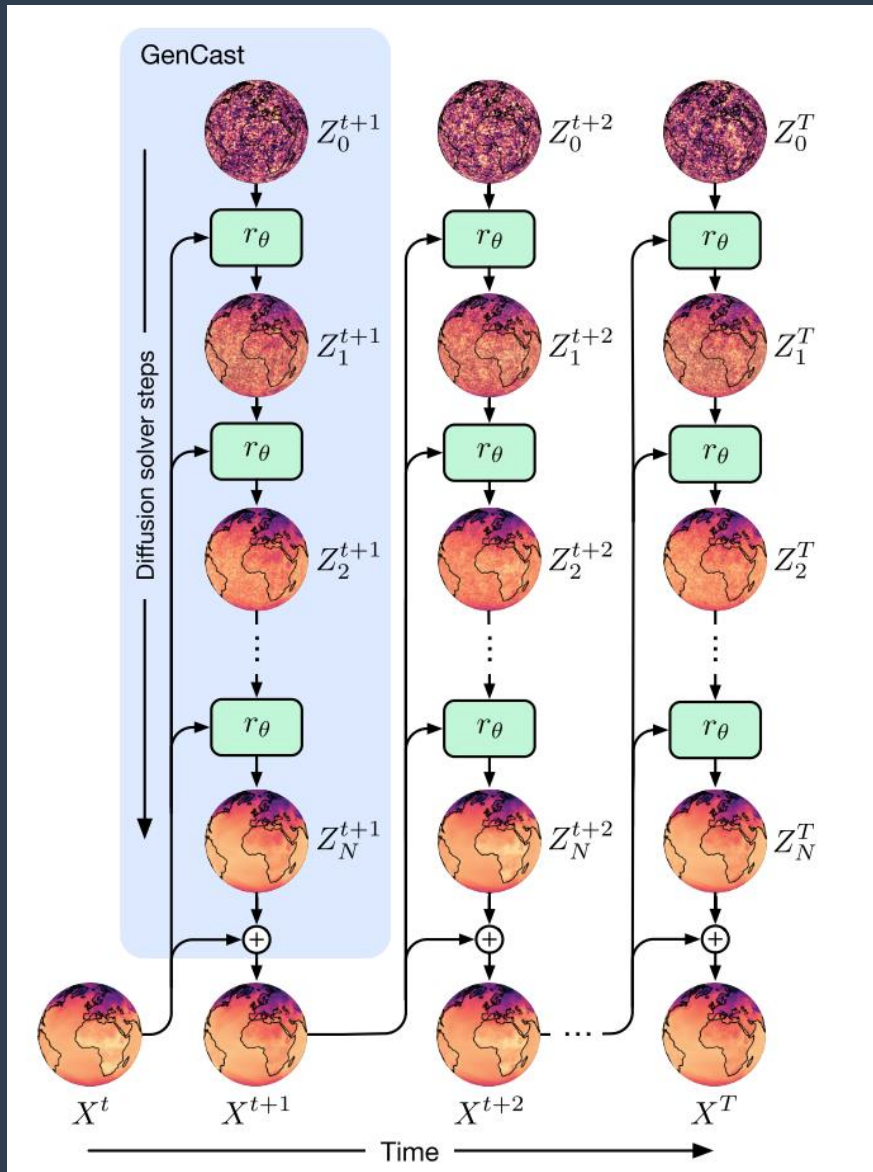
Training data is generated by adding noise in a sequence of T steps to destroy the original data. This can be seen as a sort of Markov chain



A diffusion model then learns time-stepping to reverse this process



First experimental version of AIFS-ENS and GenCast



Running in Anemoi



The screenshot shows a GitHub interface for the repository `anemoi-core`. The file path is `training / src / anemoi / training / config / diffusion.yaml`. A pull request is open, titled "feat: diffusion training (#401)" by user `ssmmnn11`, with a green checkmark indicating it is merged. The file view shows the `diffusion.yaml` file with 14 lines (12 loc) and 275 Bytes. The code content is as follows:

```
1 defaults:
2   - data: zarr
3   - dataloader: native_grid
4   - diagnostics: evaluation
5   - datamodule: single
6   - hardware: example
7   - graph: multi_scale
8   - model: graphtransformer_diffusion
9   - training: diffusion
10  - _self_
11
```

Documentation: <https://anemoi.readthedocs.io/projects/training/en/latest/user-guide/diffusion-set-up.html>

Diffusion models

Advantages:

- Neural network itself learns the uncertainties and theoretically could generate infinite number of ensemble members
- Can be used in other applications like downscaling

Disadvantages:

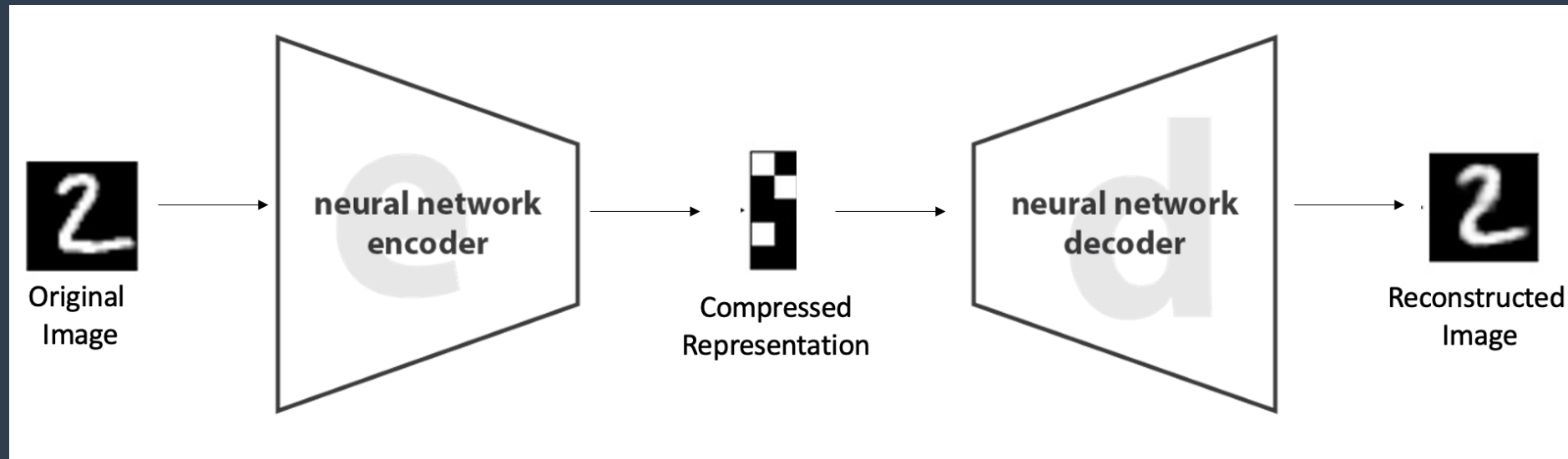
- Computationally expensive in both training and inference due to multi-stepping (we have only trained at 1 degree resolution)

Other methods that could be implemented in Anemoi

Variational Autoencoders

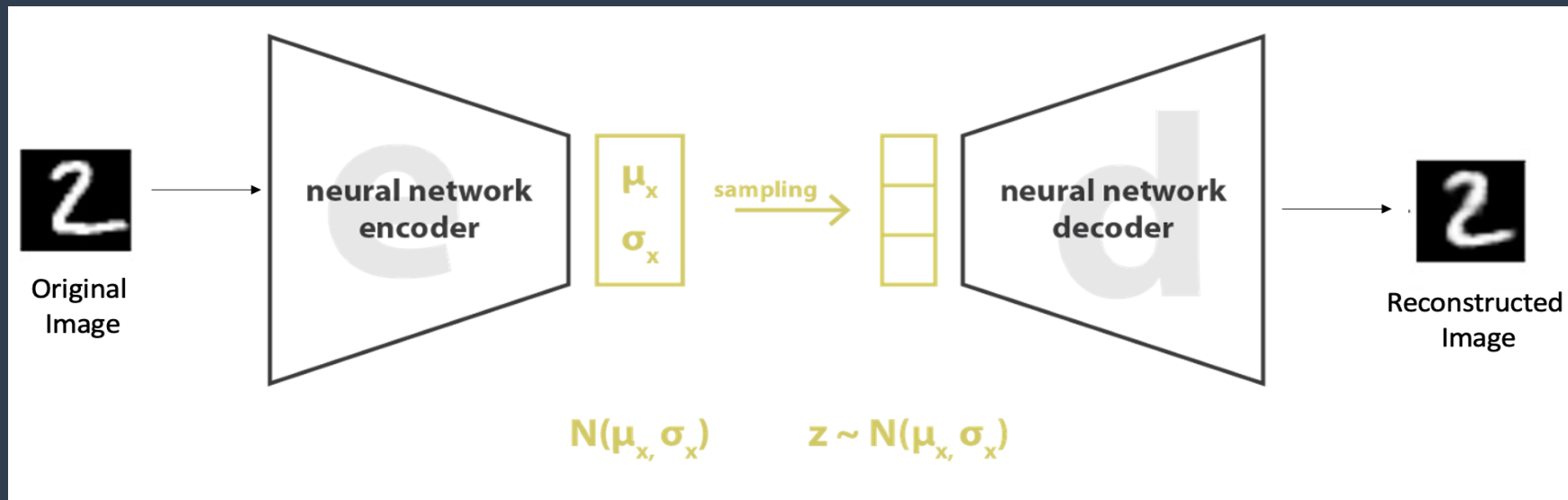
In an Autoencoder, an *encoder* encodes data from the old representation to the new one, and a *decoder* decodes the data to the old representation.

Both are neural networks trained to learn the optimum encoding and decoding



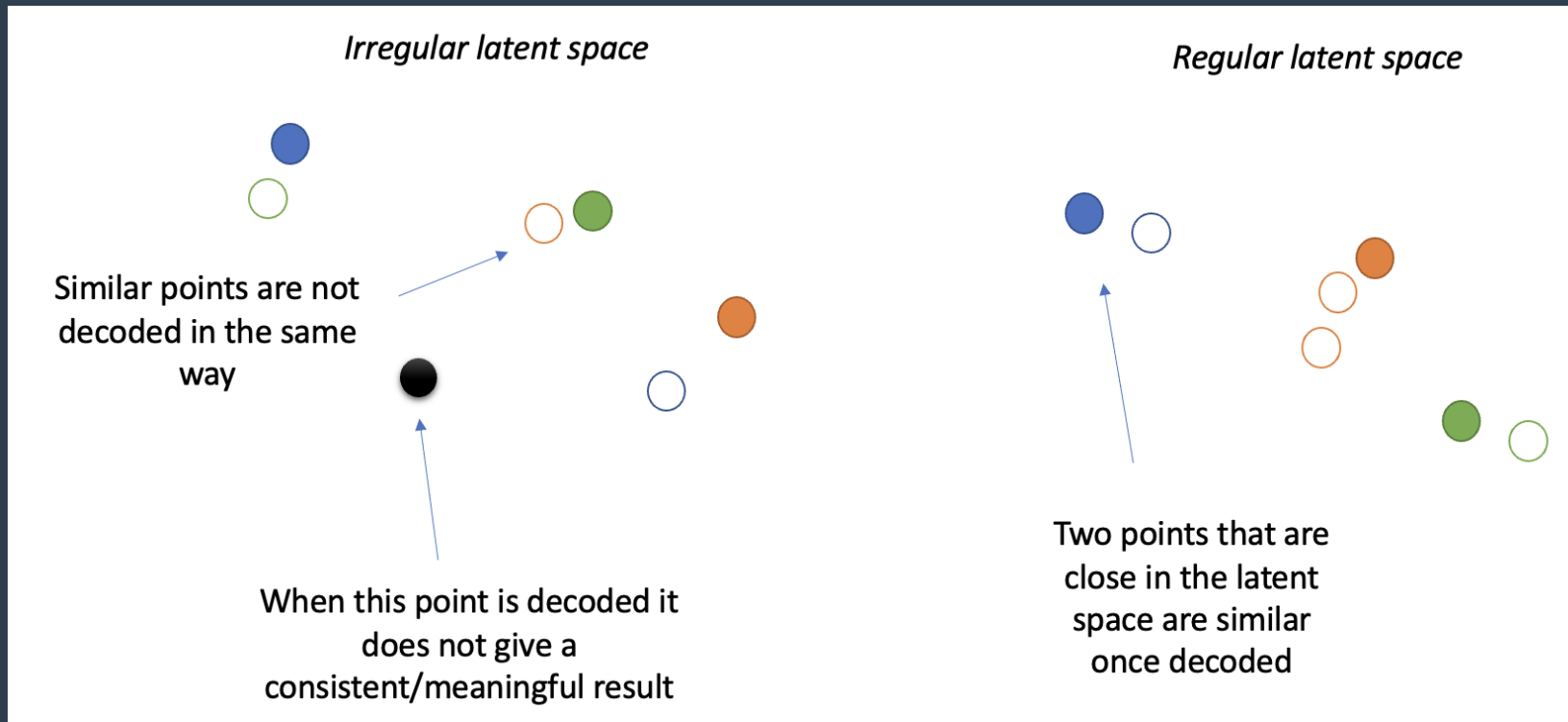
Variational Autoencoders

Variational autoencoders encode a distribution instead of a single point



Variational Autoencoders

Train the latent space so that two points close together are similar once decoded



Once trained, we can repeatedly sample from the latent space and decoding the points using the decoder and then generate an ensemble

Variational Autoencoders

Advantages:

- Can generate new ensemble members that can be used in downstream applications
- Can use deterministic loss functions instead of having to consider probabilistic ones

Disadvantages:

- Like other generative methods, ensemble members generated may not be physical
- Training costs can be high

Summary

We have considered multiple different methods to quantify uncertainty:

1. Test-time augmentation
2. Dropout Layers
3. Deep Ensembles
4. SWAG
5. Predicting parametric distributions
6. AIFS-CRPS (Injecting noise and training towards probabilistic loss)
7. Diffusion Models
8. Variational Autoencoders

They each have advantages and disadvantages, which have to be considered before deciding which one(s) is the correct one for your problem